BENNETT UNIVERSITY

School of Computer Science Engineering and Technology

Bachelor of Technology (Computer Science Engineering)

CSET400C – CAPSTONE PROJECT

MILESTONE-1 REPORT

Project Proposal, Requirements and Feasibility

JobMatcher

An AI-Powered, Explainable Job-Matching and Interview-Preparation Platform

Submitted by

Role	Name	Enrollment No.	University e-mail
Team Leader	Divyata Sangal	E23CSEU0601	e23cseu0601@bennett.edu.in
Member 2	Riya Singh	E23CSEU0935	e23cseu0935@bennett.edu.in

Group ID: 187

Project Guide: Mr Sachin Minocha

Course Coordinator: Dr. Jabir Ali

Academic Session 2026–27 (Semester VII)

Submission date: 18 October 2026

TABLE OF CONTENTS

CHAPTER 1 – PROBLEM STATEMENT, MOTIVATION AND SDG MAPPING 3
1.1 Problem Statement 3
1.2 Motivation 3
1.3 SDG Mapping and Rationale 3
CHAPTER 2 – BACKGROUND, LITERATURE AND GAP ANALYSIS 5
2.1 Background and Context 5
2.2 Approaches to Job Matching 5
2.3 Existing Solutions 6
2.4 Review of Research Literature 6
2.5 Gap Analysis 7
CHAPTER 3 – REQUIREMENTS ANALYSIS 8
3.1 Stakeholder Analysis 8
3.2 User Requirements 8
3.3 Functional Requirements 9
3.4 Non-Functional Requirements 9
CHAPTER 4 – TECHNOLOGY AND TOOL SELECTION 11
CHAPTER 5 – FEASIBILITY ANALYSIS 12
5.1 Technical Feasibility 12
5.2 Economic Feasibility 12
5.3 Operational Feasibility 13
5.4 Legal and Ethical Feasibility 13
CHAPTER 6 – SCOPE AND LIMITATIONS 14
6.1 Scope 14
6.2 Limitations 14
CHAPTER 7 – RISK ANALYSIS AND MITIGATION 15
CHAPTER 8 – PROPOSED SOLUTION AND METHODOLOGY 16
8.1 Solution Overview 16
8.2 System Architecture 16
8.3 Matching Methodology 16
8.4 Data and Evaluation Plan 17
CHAPTER 9 – EXPECTED OUTCOMES 18
REFERENCES 19

CHAPTER 1 – PROBLEM STATEMENT, MOTIVATION AND SDG MAPPING

1.1 Problem Statement

Students and recruiters lack a single, intelligent and transparent platform that matches candidates to jobs on the basis of skills and experience, explains why a match was made, and at the same time helps candidates improve their employability through resume building and interview practice.

The problem has four parts:

- **Discovery gap:** keyword search returns many irrelevant listings and misses relevant ones that use different wording (for example “ML” versus “machine learning”).
- **Screening overhead:** recruiters read large numbers of unstructured resumes and get no quick, explainable ranking against a job’s requirements.
- **Preparation gap:** students receive little personalised feedback on their resumes and interview readiness before they apply.
- **Fragmented workflow:** profile creation, resume building, job search, application tracking and interview practice are spread over separate tools.

1.2 Motivation

Socially, youth unemployment and under-employment are persistent concerns, and graduates from smaller institutions often have limited access to career guidance. A free tool that improves the quality of matching and offers on-demand interview practice supports equal access to employment. Technically, the project lets the team apply a full-stack web architecture, database design, REST APIs, NLP/ML techniques and third-party AI services in one coherent undergraduate-scale system, which is the meaningful technical component expected of a CSE capstone.

1.3 SDG Mapping and Rationale

The framework requires mapping to at least one and encourages at most two SDGs where genuinely relevant. JobMatcher is therefore mapped to **SDG 8** and **SDG 4** as its primary goals; SDG 9 and SDG 10 are mentioned only as supporting contributions.

Table 1.1: SDG mapping

SDG	Target	How JobMatcher contributes
SDG 8 – Decent Work and Economic Growth (primary)	8.5 Full and productive employment; 8.6 Reduce the share of youth not in employment, education or training	Cuts the time needed to connect young job seekers with relevant openings; gives small recruiters a free, ranked and explained shortlist.
SDG 4 – Quality Education (primary)	4.4 Increase the number of youth with relevant skills for employment	Skill-gap feedback (missing skills for a target role), resume builder and AI mock interviews help students acquire and show employable skills.
SDG 9 – Industry, Innovation and Infrastructure (supporting)	9.5 Enhance technological capability and innovation	Applies NLP/ML to a practical hiring problem with an open, modular, low-cost architecture.
SDG 10 – Reduced Inequalities (supporting)	10.2 Social and economic inclusion of all	Skills-based, explainable matching that excludes protected attributes aims to give candidates of all backgrounds a fairer chance.

CHAPTER 2 – BACKGROUND, LITERATURE AND GAP ANALYSIS

2.1 Background and Context

Online job portals are now the main route through which students and early-career professionals find work. Platforms such as LinkedIn, Naukri and Indeed list millions of vacancies, yet the experience of a fresh graduate remains difficult: jobs are spread over many sites, search is keyword-driven, resumes must be tailored role by role, and interview preparation happens elsewhere, if at all. On the other side, each opening attracts many applications of uneven quality and recruiters, especially in small firms and start-ups, shortlist manually.

Advances in natural language processing (NLP) allow a resume and a job description to be compared by meaning rather than exact words, and large language models (LLMs) can generate realistic interview questions and feedback. Combining these with a simple web application can reduce effort on both sides of hiring. This project builds on that opportunity.

2.2 Approaches to Job Matching

Recruitment technology can be grouped into four approaches:

- **Keyword and boolean search.** Listings are matched on exact or stemmed terms. It is simple and fast but ignores meaning, so synonyms and related skills are missed.
- **Content-based matching.** Resumes and job descriptions are turned into vectors, using TF-IDF (Salton and Buckley, 1988) or embeddings (Mikolov et al., 2013; Reimers and Gurevych, 2019), and compared with cosine similarity. No interaction history is needed, which suits a new platform (the cold-start case).
- **Collaborative and hybrid recommendation.** Past interactions (clicks, applications, hires) are used to learn preferences. These need large histories and can reinforce existing bias.
- **Deep person–job-fit models.** Neural networks jointly model job requirements and candidate experience (Zhu et al., 2018). They are accurate with large labelled datasets but data-hungry and hard to explain.

For an undergraduate project with no interaction data, a hybrid of structured skill matching and semantic similarity is the most practical: it works from day one, is cheap to run and can explain its output.

2.3 Existing Solutions

Table 2.1: Comparison of existing solutions (qualitative, based on publicly visible features)

Platform	Strengths	Limitations relevant to this project
LinkedIn Jobs	Very large network and listings; profile-based recommendations; Easy Apply.	Closed ranking with no explanation; some features paid; no integrated resume critique or mock interview for free users.
Naukri.com	Strong presence in India; recruiter resume database.	Keyword-heavy search; resume services are paid; limited interview practice.
Indeed	Aggregates many listings; simple search.	Generic matching; noisy results; no skill-gap feedback.
Internshala	Focus on internships and fresher roles.	Limited recruiter-side screening tools; no AI interview module.
Resume builders (Canva, Zety)	Polished templates.	Not connected to job search or applications; good templates often paid.
AI interview-practice tools	Question generation and feedback.	Stand-alone; not linked to the user's profile, target job or application status.
Proposed JobMatcher	One platform for students and recruiters; explainable score with matched/missing skills; resume builder and interview practice on one profile; low cost.	Small job inventory at start; depends on third-party AI API availability (see Chapter 7).

2.4 Review of Research Literature

Term weighting. Salton and Buckley (1988) showed that weighting terms by frequency and rarity (TF-IDF) is an effective, inexpensive way to compare documents. It remains a strong baseline for resume–job matching and is available in scikit-learn (Pedregosa et al., 2011), which the project already uses.

Distributed representations. Mikolov et al. (2013) introduced word embeddings in which similar words lie close together; Devlin et al. (2019) introduced BERT, a contextual language model; Reimers and Gurevych (2019) adapted such models to Sentence-BERT, which gives sentence embeddings comparable by cosine similarity. These are directly suited to comparing a candidate profile and a job description by meaning.

Person–job fit. Zhu et al. (2018) learn joint representations of postings and resumes to predict fit, confirming that text matching is a sound basis for recruitment while also showing the need for large historical datasets that a new platform does not have.

Fairness in automated hiring. Bogen and Rieke (2018) examine how hiring algorithms can reproduce historical inequality, and Dastin (2018) reports a recruiting tool withdrawn after showing bias against women. These motivate two design choices here: protected attributes are excluded from model inputs, and each score is explained so that a human recruiter takes the final decision.

LLMs for education and assessment. LLMs can generate domain-specific questions and formative feedback, but their output can be wrong. They are therefore used for interview practice, where the cost of an imperfect answer is low, and not as the sole decision-maker for ranking candidates.

2.5 Gap Analysis

Table 2.2: Gap analysis

#	Gap in existing solutions	How this project addresses it
G1	Matching is keyword-based or opaque; users cannot see why something was ranked.	Hybrid matching with a breakdown: matched skills, missing skills, experience fit.
G2	Job search, resume building and interview practice are separate tools.	One platform sharing a single student profile across modules.
G3	Skill-gap feedback is rare or paid.	Every match lists missing skills; the interview module can target them.
G4	Small recruiters lack affordable screening aids.	Free ranked, explained applicant list per job.
G5	Research prototypes of person–job fit are not packaged for end users.	End-to-end web application with a documented evaluation against a baseline.
G6	Fairness and privacy are rarely visible to users.	Protected attributes excluded; consent-based data use; human final decision.

Summary. The building blocks (TF-IDF, sentence embeddings, scikit-learn, LLM APIs) are mature and accessible, and large platforms leave an opening for an integrated, explainable, low-cost tool for students and small recruiters. The contribution of this project is not a new algorithm but a well-engineered, evaluated and transparent integration for a specific user group.

CHAPTER 3 – REQUIREMENTS ANALYSIS

3.1 Stakeholder Analysis

Table 3.1: Stakeholders

Stakeholder	Role / interest	Key needs	Influence
Students / job seekers (primary)	Search jobs, apply, build resume, practise interviews.	Relevant suggestions, easy profile, skill feedback, free access.	High
Recruiters / HR (primary)	Post jobs, review applicants, schedule interviews.	Ranked, explainable shortlist; less screening time; simple posting.	High
Placement cell	Supports campus placements.	Visibility of student readiness; possible institutional use.	Medium
Guide and evaluators	Supervision and assessment.	Feasible scope, technical depth, documentation.	High
Development team	Design, build, test.	Achievable plan within the semester.	High
Service providers (database, AI API, e-mail)	Supply infrastructure.	Fair-use compliance, stable API usage.	Medium
Regulators / society	Data protection, fair hiring.	Privacy, non-discrimination, transparency.	Medium

3.2 User Requirements

Students

- Register and log in securely and maintain a profile (name, headline, skills, GitHub/LinkedIn links, work preference).
- Browse, search and apply to jobs; receive a ranked list of recommended jobs with an explanation.
- Build and download a professional resume; practise interviews with AI-generated questions and feedback.
- Track the status of each application (applied, interviewed, offered, rejected).

Recruiters

- Register and log in as a recruiter; create, edit and view job postings with skills, experience, type, salary and deadline.

- View applicants per job, ranked by match score with an explanation; update application status and notify candidates by e-mail.

3.3 Functional Requirements

Table 3.2: Functional requirements (MoSCoW priority)

ID	Requirement	Priority
FR1	Students and recruiters register and log in with separate roles; passwords are stored hashed.	Must
FR2	Students create and edit a profile including skills, experience and external links.	Must
FR3	Recruiters create, edit and list job postings (title, company, location, description, skills, experience, type, salary, deadline).	Must
FR4	Students search and filter jobs and apply to a job.	Must
FR5	The system records each application and its status and shows it to both parties.	Must
FR6	Extract text from an uploaded PDF resume and populate or update the student's skills.	Should
FR7	Compute a match score per student–job pair from skills, semantic similarity, experience and preferences, and show matched and missing skills.	Must
FR8	Provide recruiters with a ranked applicant list for each job.	Must
FR9	Generate a resume from profile data in at least three templates, downloadable as PDF.	Should
FR10	Generate role-specific interview questions and feedback on answers.	Should
FR11	Send e-mail notifications for key events such as a status change.	Could
FR12	Import profile information from a LinkedIn URL through a permitted data source.	Could

3.4 Non-Functional Requirements

Table 3.3: Non-functional requirements

Category	Requirement
Performance	Typical page load under 3 s; ranking one job against up to 500 applicants under 5 s.
Security	bcrypt-hashed passwords; server-verified sessions (signed JWT or secure cookie) with role checks on every protected route; secrets only in server-side environment variables; input validation on all API routes.
Privacy	Collect only data needed for matching; explicit consent for resume processing; users can edit or delete their data; protected attributes excluded from the model.
Usability	Responsive interface for desktop and mobile; consistent navigation; clear error messages.

Category	Requirement
Reliability	Graceful degradation when the AI or ML service is unavailable (fall back to keyword matching with a clear message).
Explainability	Every match score is accompanied by its main contributing factors.
Maintainability / scalability	Modular code (API routes, schemas, ML service); Git version control; stateless web tier and separate ML microservice that scale independently.
Portability	Runs on commodity cloud or local machines; configuration through environment variables.

CHAPTER 4 – TECHNOLOGY AND TOOL SELECTION

The stack was chosen on three criteria: suitability for the requirements, learning cost for the team, and zero or low licence cost. Most components are already used in the existing prototype.

Table 4.1: Technology selection and justification

Layer	Choice	Justification	Alternatives considered
Frontend	Next.js 15 (App Router), React 19, Tailwind CSS, DaisyUI, Framer Motion	Server and client rendering in one framework; file-based routing; fast development; responsive styling.	React + Vite (needs separate backend); Angular (steeper learning curve).
Backend API	Next.js API routes (Node.js)	Same codebase as frontend; simple deployment; adequate for CRUD and orchestration.	Express.js (extra service); Django (different language).
Database	MongoDB with Mongoose	Flexible documents suit variable profile and job data (skill arrays); free Atlas tier.	PostgreSQL (strong relational model, heavier schema changes).
ML / NLP service	Python, FastAPI, scikit-learn, pandas, NumPy; sentence-transformers (planned)	Python is the ML standard; FastAPI is fast with auto API docs; scikit-learn gives TF-IDF, cosine similarity and evaluation tools.	ML inside Node.js (limited libraries); Flask (less typed validation).
LLM	Google Gemini API (gemini-2.0-flash)	Free/low-cost tier, low latency, good at question generation.	OpenAI API (paid); self-hosted open models (hardware cost).
Authentication	bcryptjs (implemented); JSON Web Tokens (planned)	Standard and stateless. The prototype keeps only a browser-side login flag, which is insecure and will be replaced.	NextAuth / OAuth.
Resume PDF / print	pdfjs-dist, react-to-print	Client-side text extraction and PDF printing at no server cost.	Server-side parsers (higher complexity).
E-mail	Resend	Simple API; free tier.	SMTP via Nodemailer.
Job data	Recruiter-posted jobs (primary); RemoteOK public API; public datasets for evaluation	Avoids terms-of-service issues of scraping logged-in sites.	Puppeteer scraping of LinkedIn (experimental script in prototype; to be removed).
Code practice / speech	JDoodle API; browser Web Speech API	Avoids building a code sandbox and a speech recogniser.	Own sandbox (high security risk).
Collaboration / testing	Git/GitHub, GitHub Projects; Jest, pytest, Postman (planned)	Traceable progress for the log diary; standard unit and API testing.	–

CHAPTER 5 – FEASIBILITY ANALYSIS

5.1 Technical Feasibility

The project is technically feasible for two undergraduates within the semester. A partial prototype already provides the web application, database schemas, hashed-password registration and login, recruiter job posting, job listing, resume builder, Gemini-based interview practice and a FastAPI microservice with a proxy route. The remaining work is bounded and uses well-documented libraries: application tracking, server-side sessions, the recruiter applicant view, and replacing the synthetic classifier with a real matching pipeline (TF-IDF first, sentence embeddings next) plus evaluation scripts and tests.

Main technical risks are manageable: embedding models may be slow on a laptop, so a small MiniLM-class model with cached embeddings will be used; public resume and job datasets are sufficient for offline evaluation; the LLM API may rate-limit, so the interview module will cache responses and fall back gracefully.

5.2 Economic Feasibility

Table 5.1: Estimated cost (approximate; third-party prices may change)

Item	Option	Estimated cost
Development tools	VS Code, Node.js, Python, Git (open source)	₹0
Database	MongoDB Atlas free tier	₹0
Web hosting	Vercel hobby tier or local deployment	₹0
ML service hosting	Local or free-tier container host	₹0 – ₹500 / month
LLM API	Gemini API free tier for development	₹0 (paid only if a pilot scales)
E-mail	Resend free tier	₹0
Domain (optional)	Annual registration	₹800 – ₹1,200 / year
Team effort	2 students × about 12 h/week × 14 weeks	Academic effort, no cash cost

Possible future revenue (not required for the capstone) includes premium recruiter features or institutional licences for placement cells while the student side stays free. The principal benefits for the capstone are educational and social, as described in the SDG mapping.

5.3 Operational Feasibility

Users need only a web browser, and the interface follows familiar job-portal conventions, so little training is needed. Operational support is limited to monitoring three external services (database, ML service, AI API). The team will operate a small pilot with a few students and one to three recruiters during Milestone-2 or the final evaluation to collect feedback.

5.4 Legal and Ethical Feasibility

- **Data protection.** Names, contact details and resumes are personal data governed in India by the Digital Personal Data Protection Act, 2023. The project will collect minimum data, obtain consent, allow correction and deletion, and use access control and encryption in transit.
- **Third-party terms.** The prototype contains an experimental Puppeteer route (app/api/scrape) that automates a browser to collect job listings. Scraping logged-in pages generally breaches platform terms and carries account and legal risk, so the proposed system will not depend on it; jobs will come from recruiter postings, open datasets and permitted APIs. The LinkedIn profile import will use only a permitted data provider or manual/PDF input.
- **Fairness.** Gender, age, religion, caste, photograph and similar attributes are not model features. Scores are advisory and explainable; the recruiter decides. A simple fairness check on test data will be reported at Milestone-2.
- **Responsible AI.** LLM output is labelled as AI-generated and used only for practice. Use of AI coding assistants will be declared per university policy.
- **Security findings in the prototype.** A code review found that the Gemini key is read from an environment variable with the NEXT_PUBLIC_ prefix, which Next.js exposes to the browser, and that the JDoodle client credentials are written directly in the source of the code-execution route. Both will be moved to server-only environment variables and the JDoodle credentials rotated before any deployment or public release.
- **Licensing.** Libraries are open source with permissive licences; datasets will be cited with their licences.

CHAPTER 6 – SCOPE AND LIMITATIONS

6.1 Scope

In scope

- Web application with student and recruiter roles, profiles, job posting, applications and status tracking.
- Hybrid matching engine (skills, semantic similarity, experience, preferences) with explanations, exposed through the FastAPI service.
- Resume text extraction from PDF, resume builder with templates, and AI interview practice.
- Offline evaluation of the matching engine on public data against a keyword baseline; basic e-mail notifications, security hardening and test cases.

Out of scope

- Native mobile applications; payments, subscriptions or paid placements.
- Video- or voice-based interview analysis (facial expression, tone) because of ethical and technical complexity.
- Large-scale scraping of third-party job sites; fully automated hiring decisions without human review.

6.2 Limitations

- The platform starts with little real data; evaluation will rely on public datasets and a small pilot, so results may not generalise to all industries.
- Matching quality depends on the text quality of resumes and job descriptions.
- LLM feedback can be inaccurate and is advisory only.
- The pilot will be small, so user-satisfaction conclusions are indicative, not definitive; free tiers of external services impose rate limits.

CHAPTER 7 – RISK ANALYSIS AND MITIGATION

Table 7.1: Risk register

ID	Risk	Likelihood	Impact	Mitigation
R1	No real labelled data for training and evaluation	High	High	Use public datasets and a small hand-labelled test set; rely on unsupervised similarity (TF-IDF/embeddings) so labels are not required for training.
R2	Scraping or profile import breaches third-party terms	Medium	High	Remove login-based scraping; use recruiter postings, datasets and permitted APIs.
R3	LLM API quota, cost or outage	Medium	Medium	Cache responses; fall back to a pre-written question bank; configurable provider.
R4	Leakage of API keys or personal data	Medium	High	Move keys to server-only variables and rotate; no secrets in the repository; HTTPS; session checks on every route.
R5	Biased or unexplainable ranking	Medium	High	Exclude protected attributes; show factor breakdown; human final decision; basic fairness check.
R6	Schedule slip or team-member unavailability	Medium	Medium	Weekly sprints with clear owners; prioritise “Must” requirements; weekly guide check-in; one-week buffer.
R7	ML service unreachable from the web app	Low	Medium	Health-check endpoint; timeout and fallback to keyword match; documented start-up order.
R8	Embedding model too slow or large for free hosting	Medium	Medium	Small model, cached embeddings; start with lightweight TF-IDF.
R9	Low participation in the validation survey	Medium	Low	Use campus channels and the placement cell; keep the survey to five minutes.

CHAPTER 8 – PROPOSED SOLUTION AND METHODOLOGY

8.1 Solution Overview

JobMatcher is a two-sided web platform. Students keep one profile and use it to search jobs, receive ranked recommendations, generate a resume and practise interviews. Recruiters post jobs, receive applications and see applicants ranked by an explainable match score. The matching logic runs in a separate Python service so that it can be developed, tested and scaled independently of the web application.

8.2 System Architecture

The system uses a layered architecture (Figure 8.1). The web application reaches the matching service over HTTP through a proxy route, which keeps the service address private and lets the web tier apply authentication and timeouts.

Presentation layer	Next.js / React pages – Student: Dashboard, Jobs, Profile, Resume builder, Interview. Recruiter: Dashboard, Create Job, Candidates.
Application layer	Next.js API routes – register/login, profile, jobs, applications, e-mail, Gemini proxy, ML proxy (/api/ml/predict).
Intelligence layer	FastAPI service – text pre-processing → skill extraction → TF-IDF / embeddings → hybrid scoring → ranked, explained results. Gemini API for interview questions.
Data layer	MongoDB (Atlas) – Users, Recruiters, UserProfiles, Jobs, Applications; cached embeddings; offline evaluation datasets.

Figure 8.1: Proposed layered system architecture

8.3 Matching Methodology

The engine computes a score from 0 to 100 for each candidate–job pair as a weighted combination of four interpretable sub-scores (Table 8.1).

Table 8.1: Proposed matching pipeline

Step	Description
1. Pre-processing	Clean text, lower-case, remove stop-words, normalise skill synonyms with a small skills dictionary (e.g. “ML” → “machine learning”).
2. Skill extraction	Match normalised text against the dictionary to obtain skill sets for the resume and the job.

Step	Description
3. Skill overlap (S_1)	Weighted coverage of the job's required skills by the candidate's skills; required skills weigh more than optional ones.
4. Semantic similarity (S_2)	Cosine similarity of TF-IDF vectors (baseline), then sentence embeddings (improved model), between resume text and job description.
5. Experience fit (S_3)	Compare years of experience / education level with the requirement; partial credit near the threshold.
6. Preference fit (S_4)	Match job type, location and remote / on-site / hybrid preference.
7. Final score	Score = $100 \times (w_1S_1 + w_2S_2 + w_3S_3 + w_4S_4)$; initial weights (0.40, 0.35, 0.15, 0.10), tuned on a validation set.
8. Explanation	Return matched skills, missing skills and the sub-score breakdown with the ranked list.

Rationale. A weighted, interpretable combination is easy to explain, needs no historical interaction data, and can later be extended: once enough application and hiring outcomes exist, a supervised model (logistic regression or gradient boosting, already supported by scikit-learn) can learn the weights. The prototype's logistic-regression endpoint is retained as the place where such a model can be plugged in.

8.4 Data and Evaluation Plan

- **Data.** Public resume and job datasets (for example from Kaggle, with licences checked and cited), a small hand-labelled set of candidate–job pairs reviewed by the guide, and synthetic or anonymised profiles for demonstration. No real user data is used for evaluation without consent.
- **Baselines.** Exact keyword overlap and TF-IDF cosine similarity.
- **Metrics.** Precision@k, Recall@k, Mean Reciprocal Rank and NDCG@k for ranking quality; response time for performance; score-distribution comparison across groups for fairness; System Usability Scale (SUS) for pilot users.
- **Success criterion.** The hybrid method should outperform the keyword baseline on NDCG@10 on the labelled set; the exact target will be fixed after a first run in Milestone-2.

CHAPTER 9 – EXPECTED OUTCOMES

Table 9.1: Expected outcomes and success measures

Outcome	Measure of success
Working two-sided web platform	All “Must” functional requirements demonstrated end to end.
Explainable hybrid matching engine	Ranked results with matched/missing skills; beats the keyword baseline on NDCG@10 and Precision@10 on the labelled test set.
Resume builder and interview module tied to the profile	Resume in at least three templates; interview sessions with feedback completed by pilot users.
Validated requirements	Survey of at least 30 students and at least 3 recruiter interviews summarised.
Secure, privacy-aware implementation	No secrets in client code or repository; protected attributes excluded; consent text present.
Evidence and documentation	Test cases, evaluation results, screenshots, weekly log diary, final report and presentation.
Societal contribution	Pilot feedback (SUS score and comments) indicating reduced search and screening effort, supporting SDG 8 and SDG 4.

REFERENCES

[1] Salton, G. and Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513–523.

[2] Mikolov, T., Chen, K., Corrado, G. and Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv:1301.3781.

[3] Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. Proc. NAACL-HLT 2019.

[4] Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. Proc. EMNLP-IJCNLP 2019.

[5] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[6] Zhu, C. et al. (2018). Person-job fit: Adapting the right talent for the right job with joint representation learning. ACM Transactions on Management Information Systems, 9(3).

[7] Bogen, M. and Rieke, A. (2018). Help Wanted: An Examination of Hiring Algorithms, Equity, and Bias. Upturn.

[8] Dastin, J. (2018). Amazon scraps secret AI recruiting tool that showed bias against women. Reuters, 10 October 2018.

[9] United Nations (2015). Transforming our world: the 2030 Agenda for Sustainable Development. https://sdgs.un.org/goals

[10] Government of India (2023). The Digital Personal Data Protection Act, 2023.

[11] Next.js documentation. https://nextjs.org/docs (accessed October 2026).

[12] FastAPI documentation. https://fastapi.tiangolo.com (accessed October 2026).

[13] MongoDB documentation. https://www.mongodb.com/docs (accessed October 2026).

[14] Google Gemini API documentation. https://ai.google.dev (accessed October 2026).

[15] Bennett University, School of CSET (2026). CSET400C Capstone Project Evaluation Framework, Academic Session 2026–27.